IMEN574 / IDSC574 · PROGRAMMING FOR AX

9

Classification

Sangwoo Mo · POSTECH
Spring 2027

Where We Are

01 **Programming with AI**

Lectures 2–4

02 **Python and Classical Machine Learning**

Lectures 5–13

03 **Deep Learning and Foundation Models**

Lectures 14–22

Outline: Classification

01 **Probability Models**

Fit scores and probabilities

02 **Decision Rules**

Trade misses against false alarms

03 **Reliable Evaluation**

Check probabilities and populations

01 · Probability Models

01 **Probability Models**

Fit scores and probabilities

02 **Decision Rules**

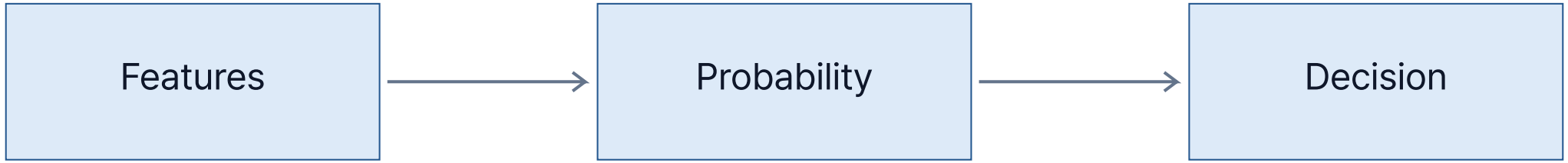
Trade misses against false alarms

03 **Reliable Evaluation**

Check probabilities and populations

Classification Workflow

A score becomes an action only after a decision rule



The model and decision rule solve different problems

Binary and Multiclass Targets

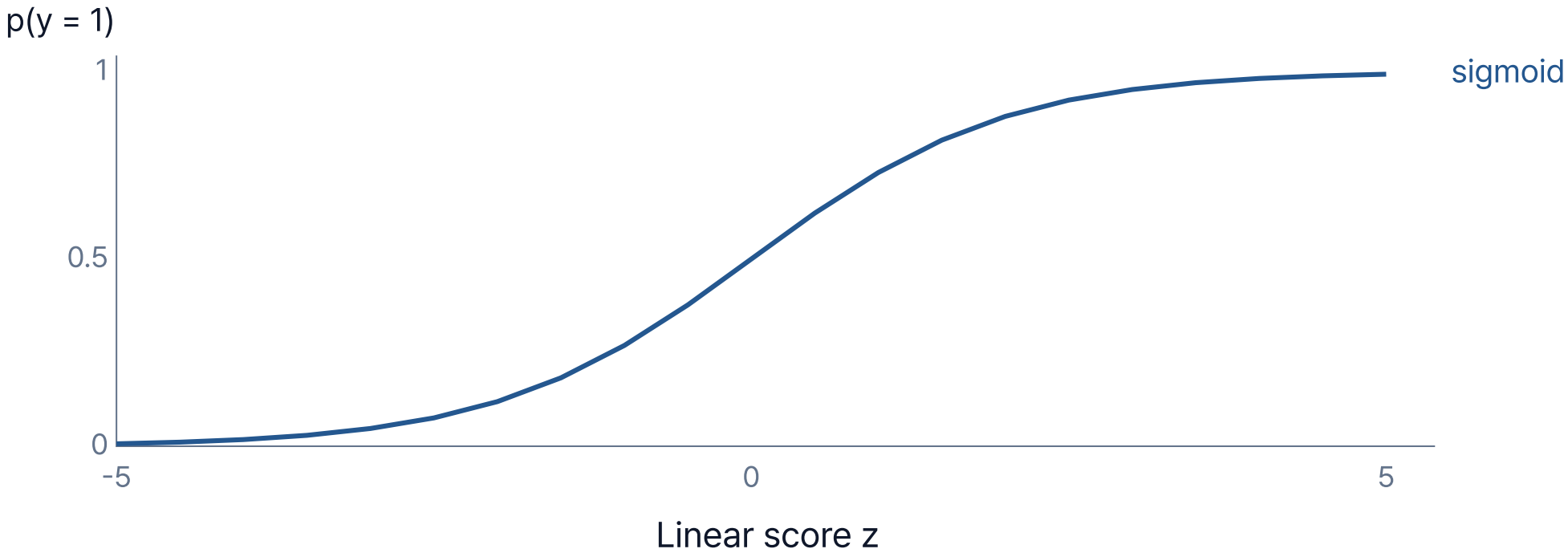
The target defines the output space

	y	Meaning
Label 0	0	normal
Label 1	1	fault

A binary label records one chosen event

Logistic Regression

A linear score is mapped into a probability

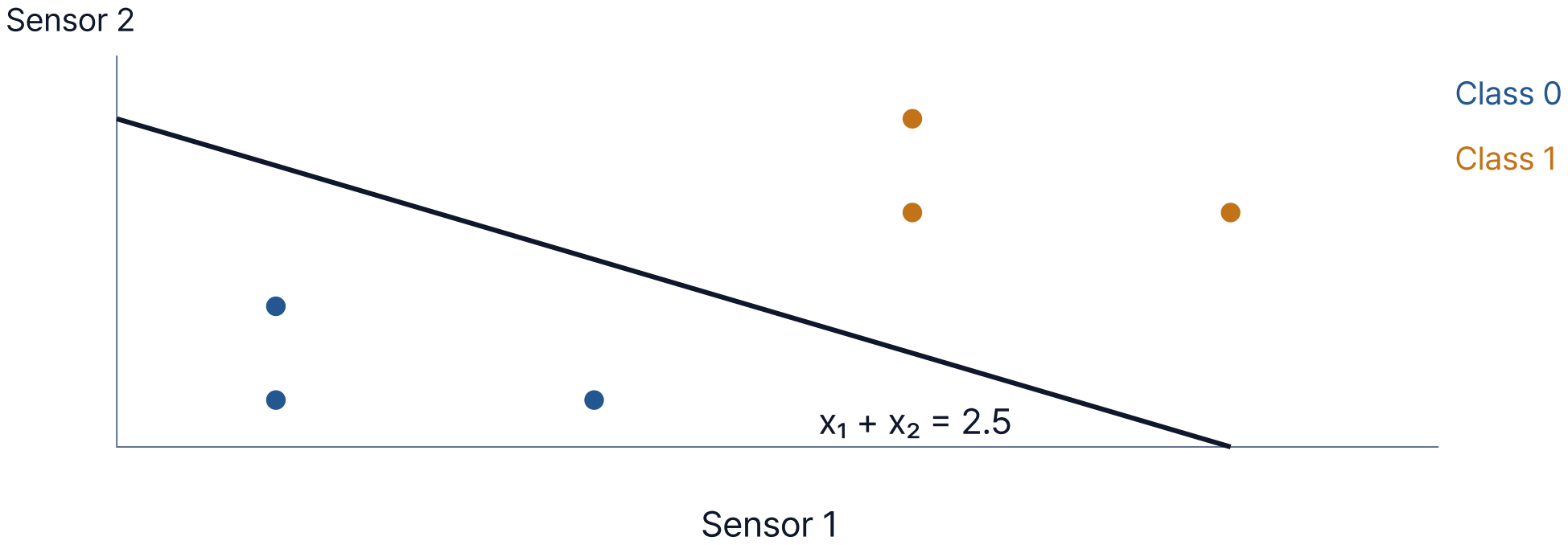


$$p = 1 / (1 + \exp(-z)), \; z = w^{T}x + b$$

Large scores approach one without exceeding it

Decision Boundary

With w = (1, 1) and b = −2.5, the boundary has $x_1 + x_2 = 2.5$

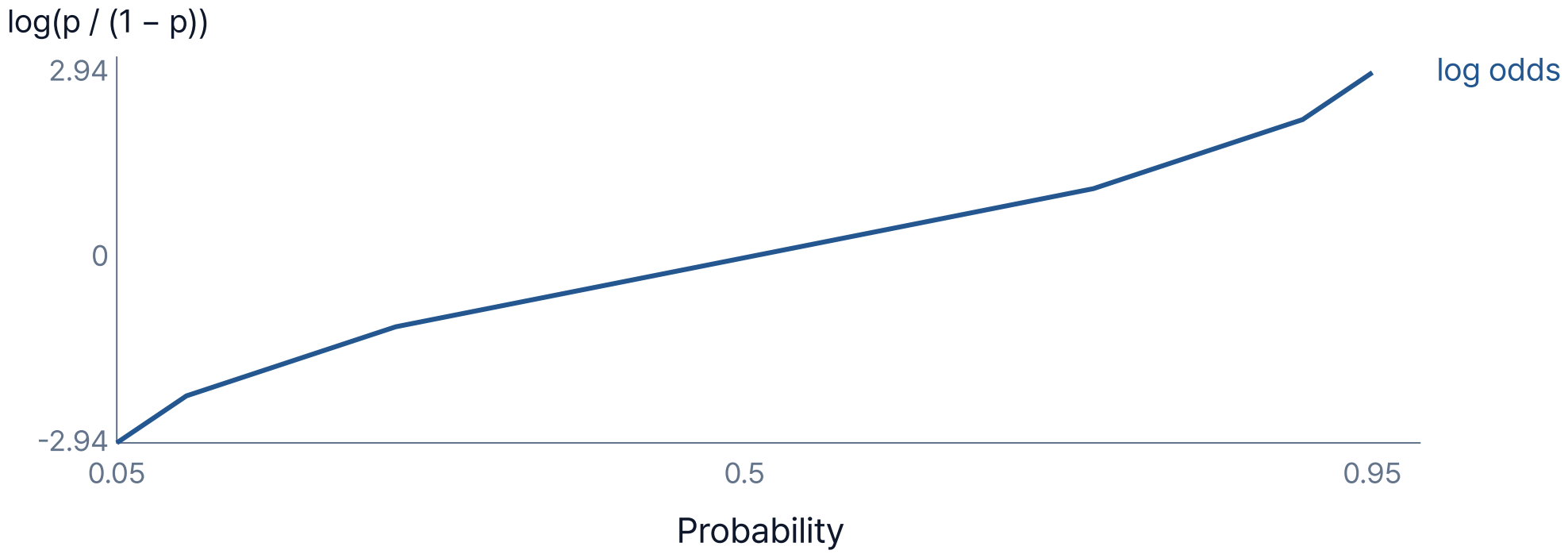


$$w^T x + b = 0 \text{ at } p = 0.5$$

A threshold selects one side of the score surface

Log Odds

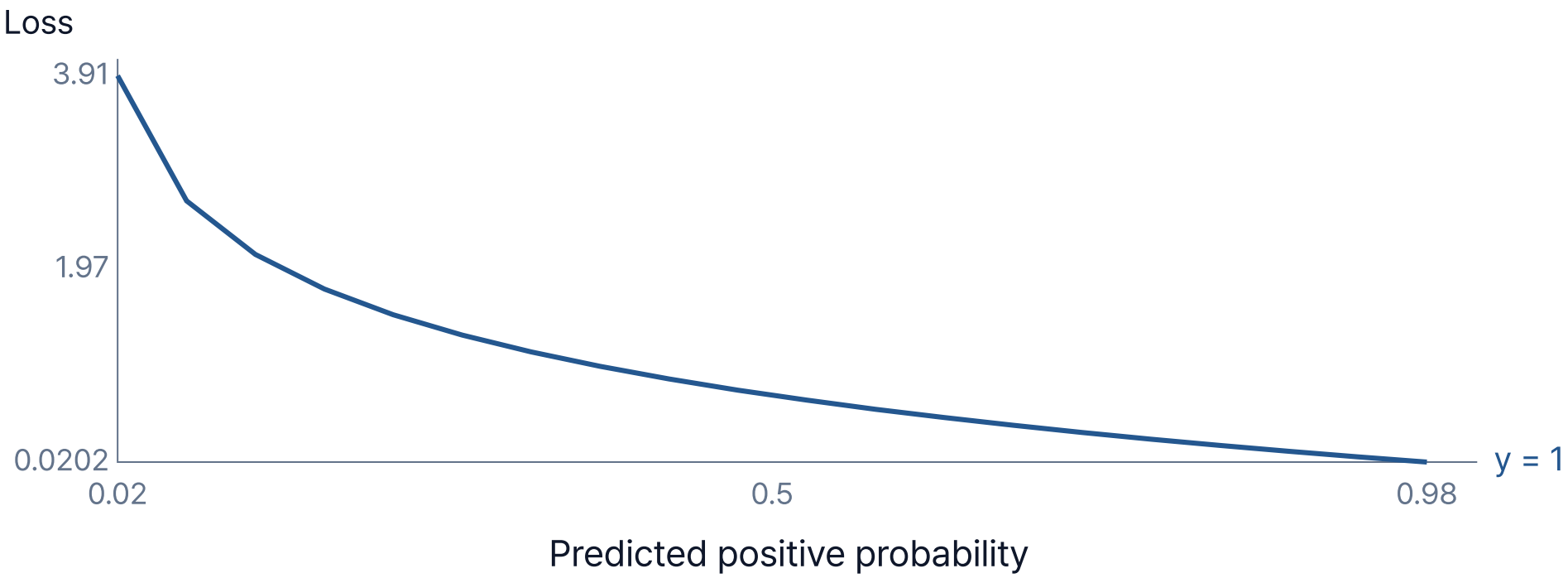
A coefficient changes odds multiplicatively



Probability changes are nonlinear even with linear log odds

Cross-Entropy Loss

Confident mistakes receive a large penalty



A positive example predicted near zero is costly

Stable Loss Calculation

Compute the loss from logits without taking log of zero

```
z = np.array([-4., 0., 4.])
y = np.array([0., 1., 1.])
loss = np.logaddexp(0, z) - y*z
print(np.round(loss, 4))
```

```
[0.0181 0.6931 0.0181]
```

Probability Gradient

The residual p − y drives each update

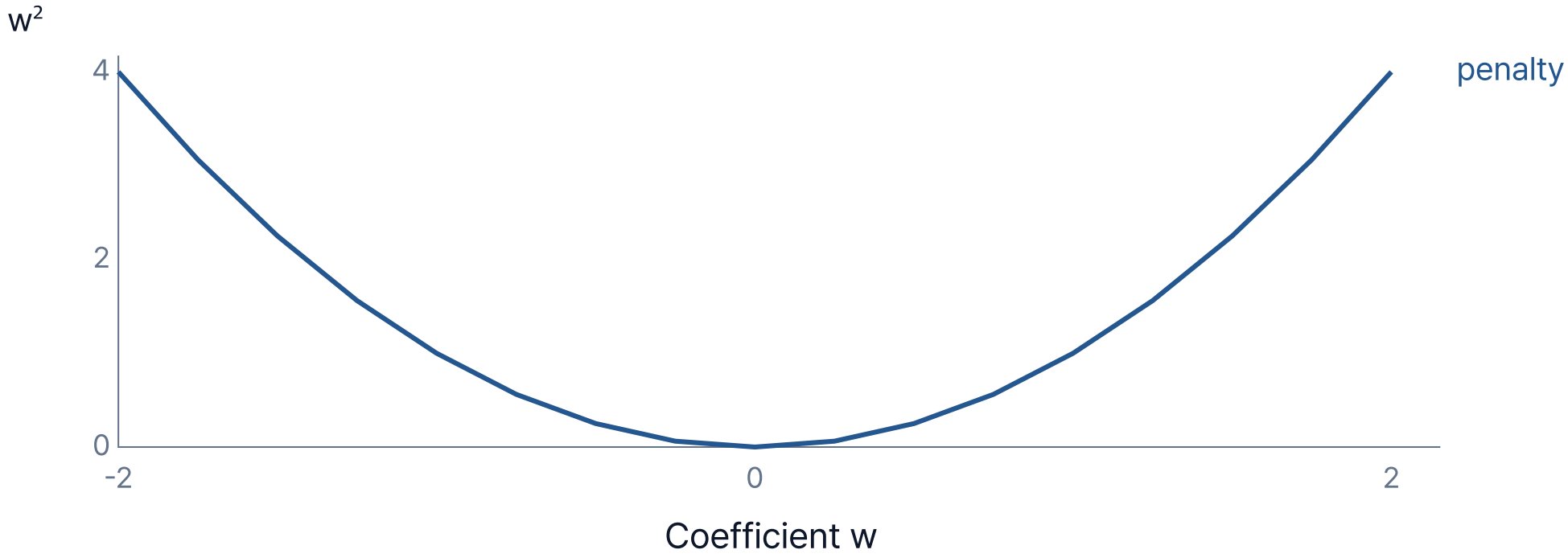
	y	p	p − y
Negative	0	0.2	0.2
Positive	1	0.2	-0.8

$$\partial L/\partial w = (p - y)x$$

The same prediction yields opposite correction directions

Regularized Classification

Penalize large coefficients to limit sensitivity



$$\text{Objective} = \text{mean cross-entropy} + \lambda \|w\|^2$$

The penalty grows with coefficient magnitude

Multiclass Softmax

Class probabilities share one normalization

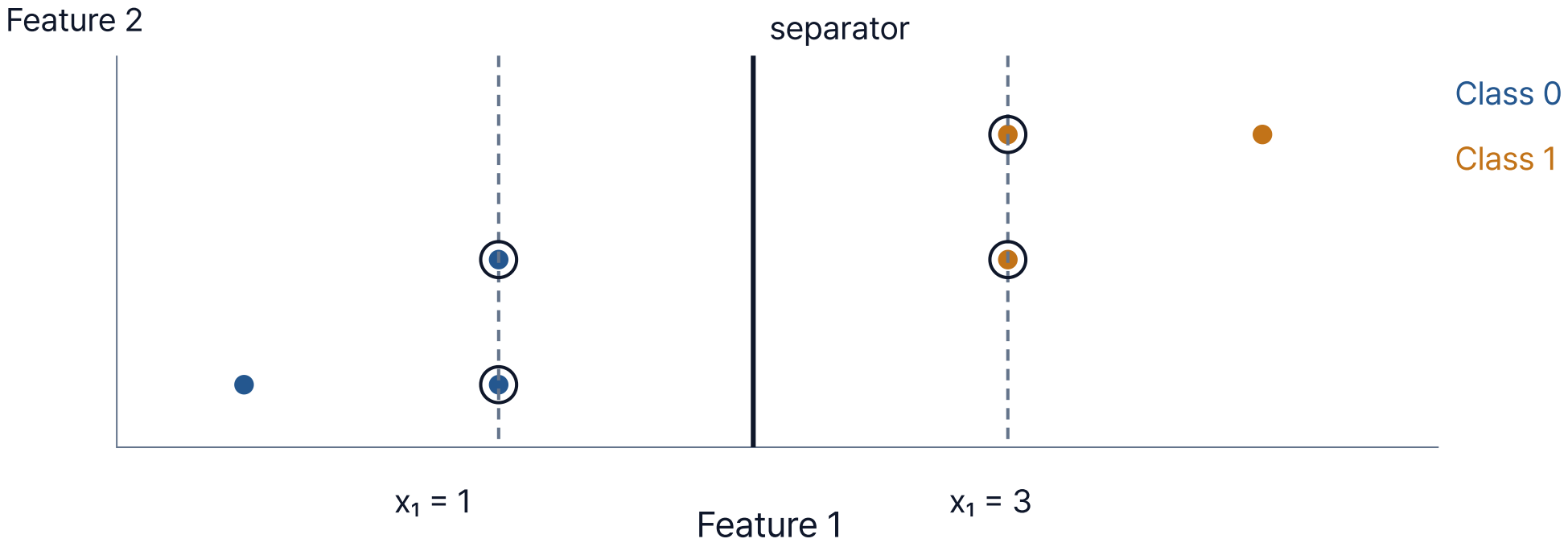
Class	Logit	Probability
A	2.0	0.665
B	1.0	0.245
C	0.0	0.09

$$p_k = \exp(z_k) / \Sigma_j \exp(z_j)$$

Increasing one logit lowers the relative share of others

Support Vector Classification

A margin separates fitting from distance to the boundary



Margin depends on the nearest training examples

The nearest points constrain the separating boundary

02 · Decision Rules

01 **Probability Models**

Fit scores and probabilities

02 **Decision Rules**

Trade misses against false alarms

03 **Reliable Evaluation**

Check probabilities and populations

Predictions at a Threshold

One score list supports many decisions

	1	2	3	4	5	6
True y	0	0	1	0	1	1
Score	0.05	0.25	0.40	0.55	0.75	0.90
$p \geq 0.5$	0	0	0	1	1	1

The third example is a missed positive at 0.5

Confusion Matrix

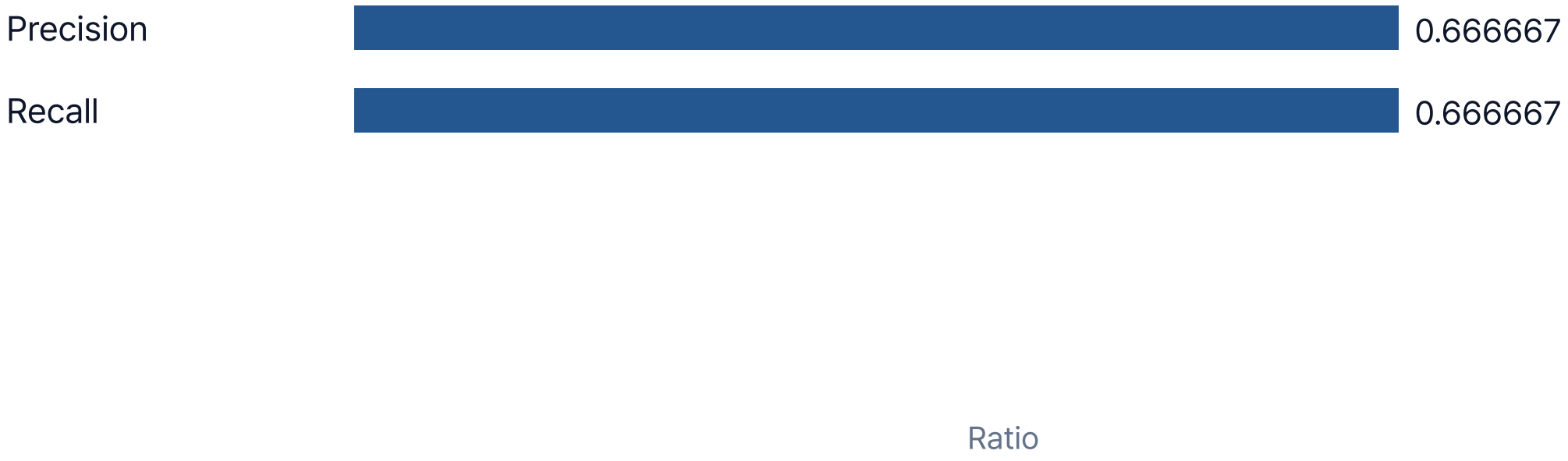
Count errors by their consequence

	Pred 0	Pred 1
True 0	2	1
True 1	1	2

One false alarm and one miss remain

Precision and Recall

Ask two different questions about positive cases



Both equal two thirds on these six examples

F1 Score

The harmonic mean penalizes an unbalanced pair

Precision	Recall	F1
1	0.25	0.4
0.5	0.5	0.5
0.25	1	0.4

$$F1 = 2PR / (P + R)$$

Perfect recall alone does not produce a high F1

Caveat: Class Imbalance

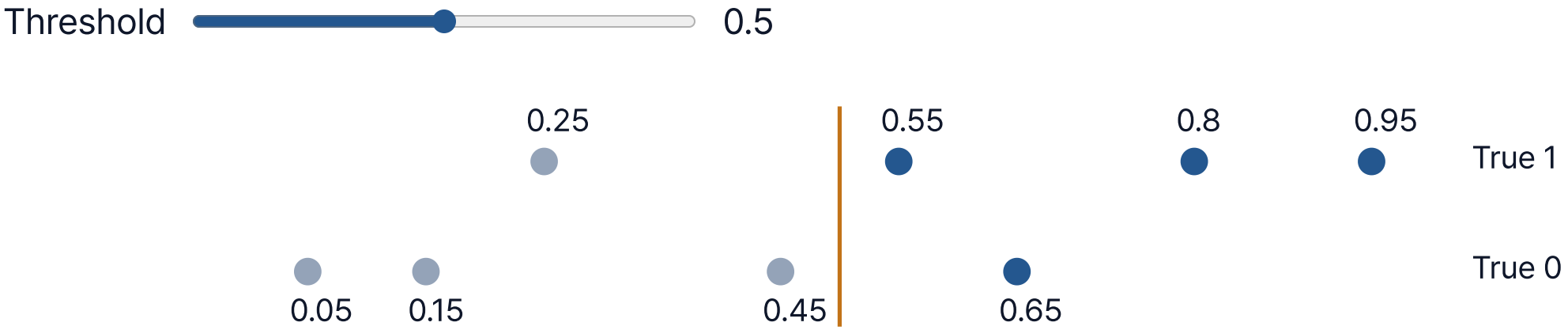
A majority predictor can hide every positive case

	Pred 0	Pred 1
True 0	990	0
True 1	10	0

Ninety-nine percent accuracy can coexist with zero recall

Demo: Decision Threshold

Change the threshold and compare false alarms with misses

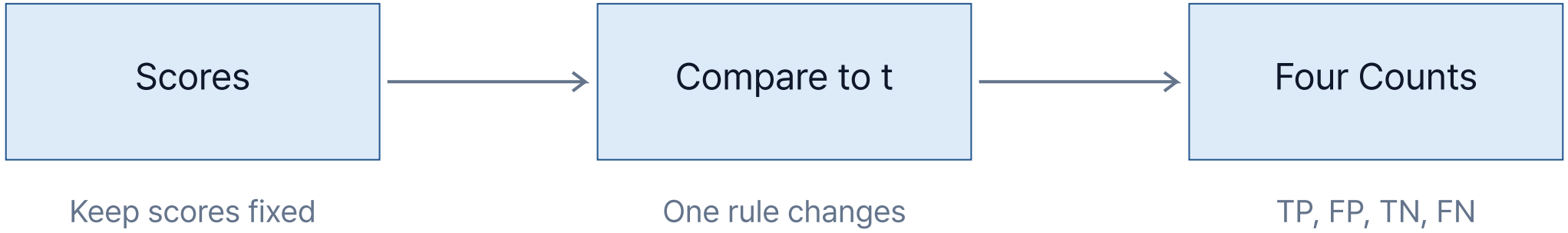


Threshold 0.50 | TP 3, FP 1, FN 1, TN 3

Precision 0.75, recall 0.75, accuracy 0.75

Answer: Threshold Accounting

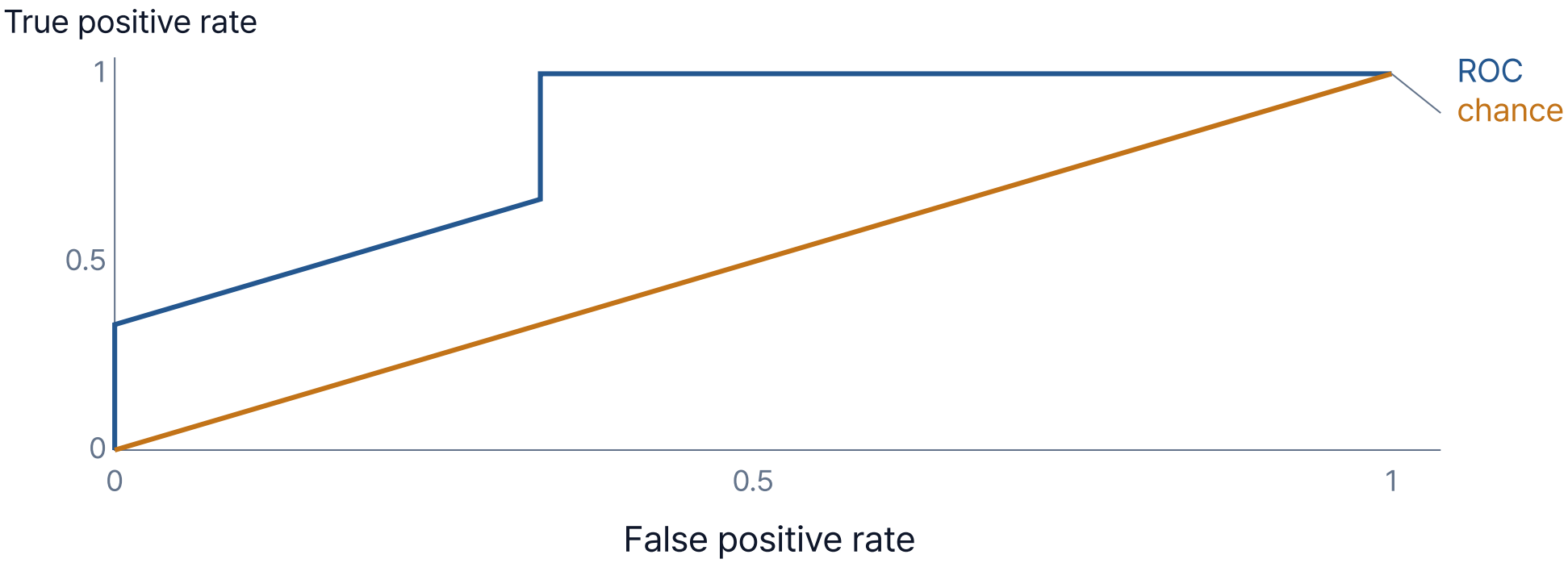
Recompute all four counts after every threshold change



Changing the threshold changes decisions rather than model parameters

ROC Space

Compare sensitivity with false-alarm rate



Thresholds trace a path through operating points

Precision-Recall Space

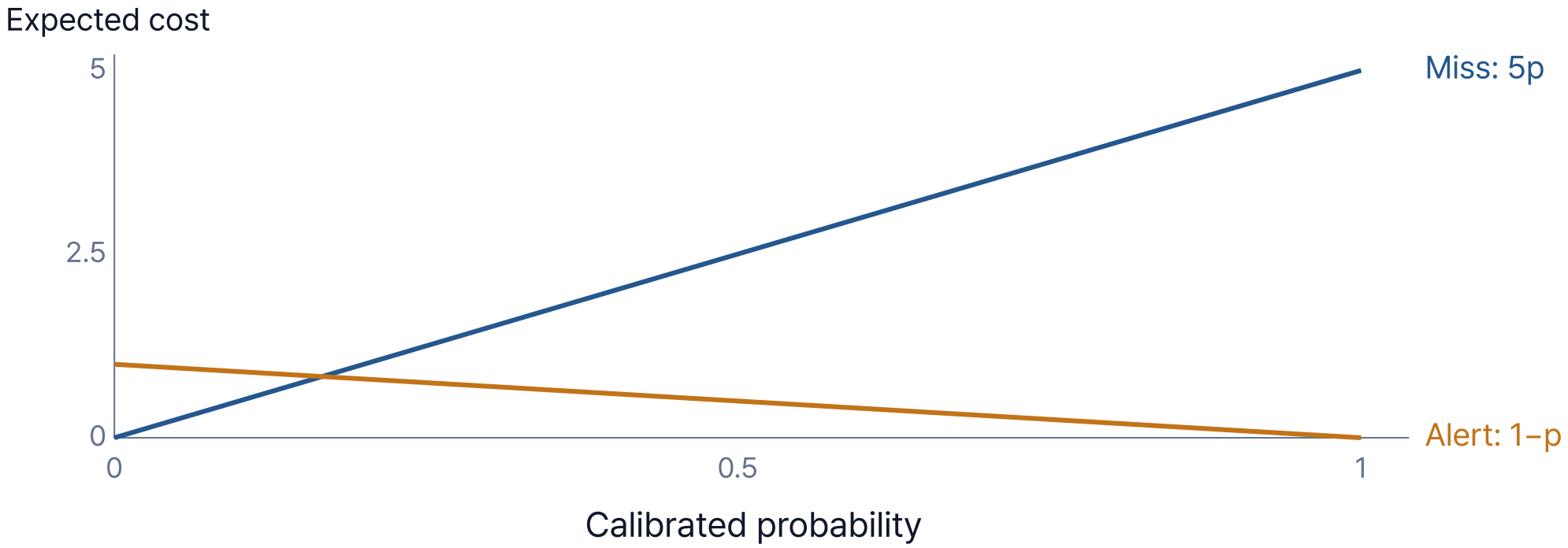
Positive prevalence changes the meaning of retrieved cases

TP	FP	Precision
10	90	0.1
10	10	0.5
10	0	1

The same ten true alerts can require very different review effort

Cost-Sensitive Decisions

Choose the action with lower expected cost

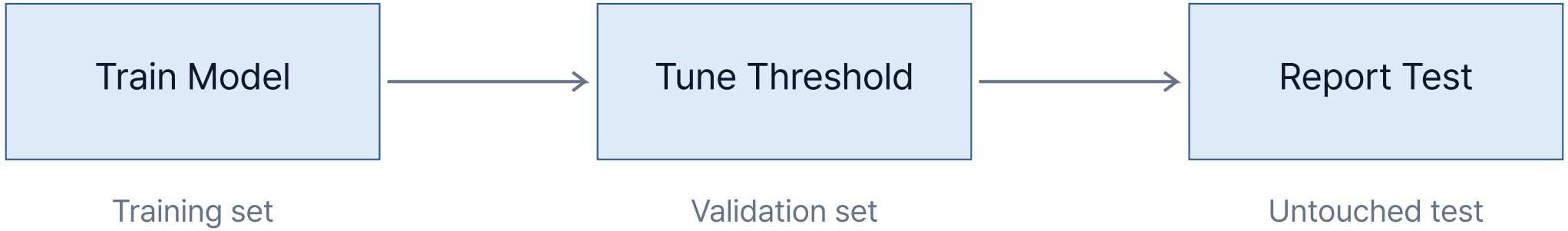



```
Alert when p > C_FP / (C_FP + C_FN)
```

Higher miss cost lowers the action threshold

Threshold Selection Split

Tune a decision rule on validation data



A threshold chosen on the test labels contaminates evaluation

03 · Reliable Evaluation

01 **Probability Models**

Fit scores and probabilities

02 **Decision Rules**

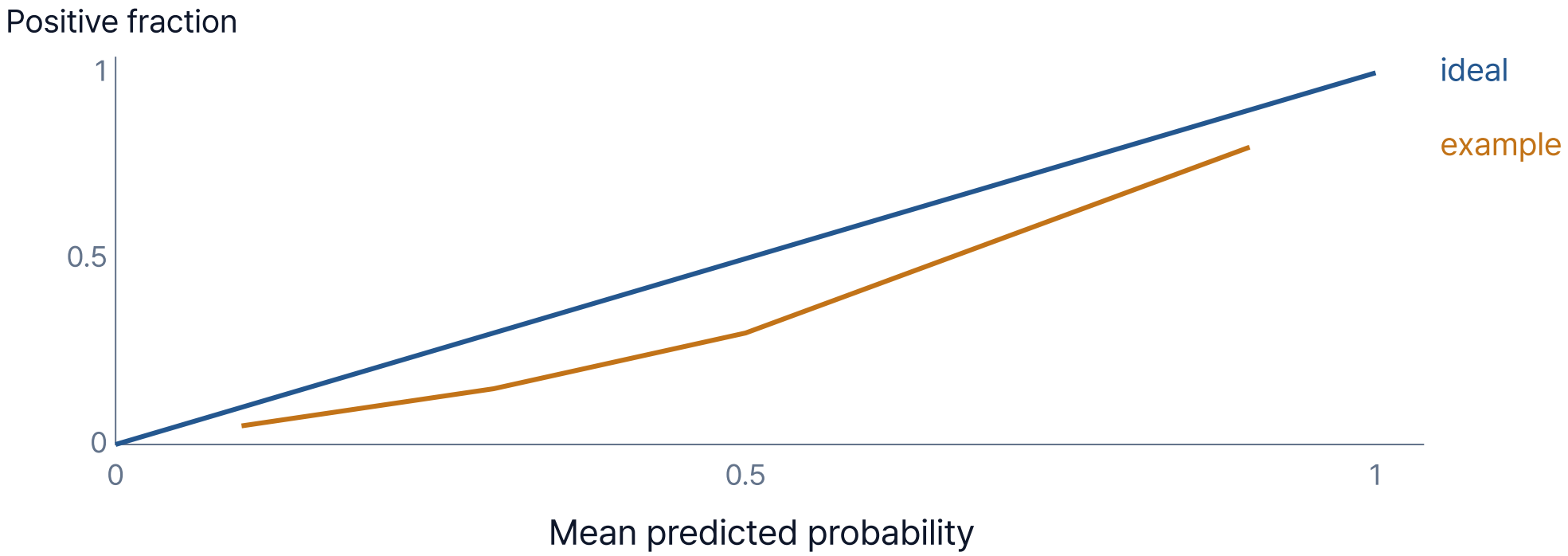
Trade misses against false alarms

03 **Reliable Evaluation**

Check probabilities and populations

Probability Calibration

Predicted probabilities should match observed frequencies



This example predicts more positives than each bin observes

Brier Score

Measure squared probability errors rather than only labels

y	p	$(p - y)^2$
1	0.9	0.01
1	0.6	0.16
0	0.6	0.36

The score distinguishes predictions with the same hard label

Uncertainty Decomposition

Confidence can fail for different reasons

Source	Example	Response
Data noise	Ambiguous label	Improve measurement
Limited coverage	New machine	Collect new evidence
Model mismatch	Linear boundary	Compare alternatives

A confident score does not identify its source of error

Stratified Validation

Preserve rare labels when estimating ordinary random splits

	Class 0	Class 1
Fold A	18	2
Fold B	18	2
Fold C	18	2

Each example fold contains both classes

Group-Aware Evaluation

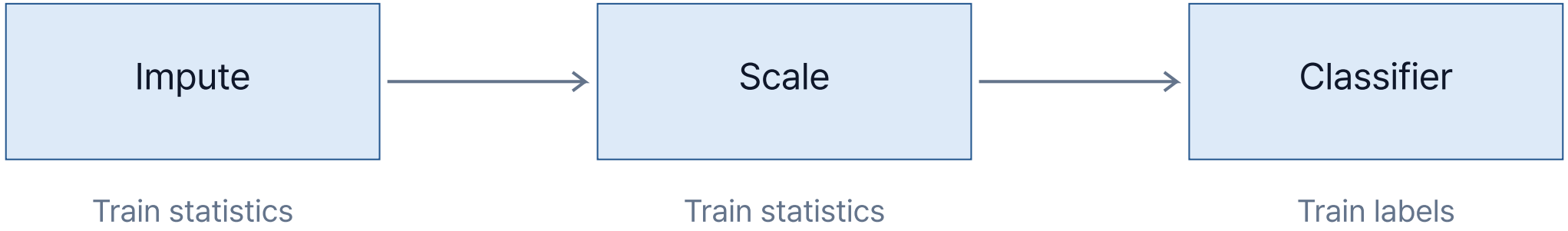
Keep examples from one entity on one side

	Entity 1	Entity 2	Entity 3
Train	A	A	B
Test	C	C	D

Repeated measurements require an entity boundary

Classification Pipeline

Fit every learned transformation inside the training split



Validation rows pass through fixed transformations

Auditing Generated Predictions

Verify the positive label and metric inputs explicitly

```
y = np.array([0, 0, 1, 0, 1, 1])
p = np.array([.05, .25, .4, .55, .75, .9])
a = p >= .5
print("TP", int(((y==1)&a).sum()))
print("FP", int(((y==0)&a).sum()))
print("FN", int(((y==1)&~a).sum()))
```

```
TP 2
FP 1
FN 1
```

Class Weighting

Changing the loss changes what fitting emphasizes

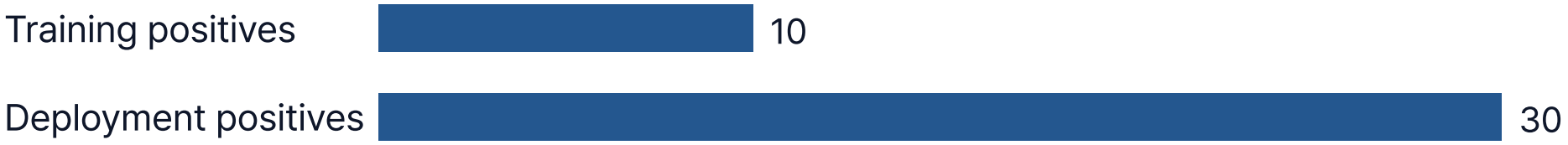


Loss multiplier

A weighted positive error contributes five times as much

Distribution Shift

A fixed rule can meet a different positive rate



Cases per 100

The event becomes more common in the second population

Model Selection Record

Retain the assumptions behind the selected operating point

Item	Recorded Choice
Positive class	Fault requiring inspection
Split unit	Machine
Primary metric	Recall at review budget
Threshold	Chosen on validation
Final report	Frozen test once

An operating point is meaningful only with its evaluation protocol

Takeaway: Classification

01 Separate probability estimation from action selection

02 Tune thresholds using validation data

03 Audit rare events and deployment shifts